# PDFalyzer

## AcroForm Field Showcase

Comprehensive test document — every interesting variant, quirk, and edge case

Each box shows a live form field above an explanation of what is unusual and what breaks.

This PDF was generated by FormFieldShowcasePdfGenerator (Apache PDFBox 3.x). It contains real AcroForm widgets with every interesting flag combination, unusual /DA string, border style, rotation, and signature variant.
Use it to stress-test PDF parsers, form renderers, and PDF editing tools.

## Contents


| | |
|---|---|
| Text Fields | Required • Read-Only • Multiline • Password • Comb • Auto-Size Font • Giant Font • Alignment • Rich Text • MaxLen |
| Checkboxes & Radio Buttons | Standard • Custom On-State • Coloured Background • Radio Group • NoToggleToOff |
| Combo & List Boxes | Non-Editable • Editable • Paired Options • Multi-Select • TopIndex • Value Not in Opt |
| Push Buttons | Reset Form • Submit Form • Print-Only (NoView) |
| Widget Border Styles | No Border • Dashed • Beveled • Inset • Underline • 90° Rotation • MK Colours |
| Signature Fields | Empty Placeholder • Lock Dictionary • DocMDP Certification |
| Extreme Edge Cases | Hidden Widget • Zero-Height • 1000-char Value • Hierarchical Name • Missing DA • V/DV Mismatch |


Embedded font: FreeSans (GNU FreeFont). All AcroForm fields reference this font via the /DR /Font resource dictionary.
Form /NeedAppearances = true. Open in Adobe Acrobat to regenerate appearances; some viewers may render fields differently.

# TEXT FIELDS

PDTextField — Standard & Unusual Variants

## TEXT FIELDS — 6 Visual Themes (Plain / Green / Amber / Red / Gray / Purple)

Plain — no MK, no colour (baseline)

Plain default (no styling)

Green — success state

Valid entry [OK]

Amber — warning state

Check this value!

Red — error / invalid

Invalid input [ERR]

Gray — disabled / read-only

Read-only value

Purple — custom accent

Custom accent

## TEXT — Required Field (Ff bit 2 = Required)

Jane Doe ← required field

The Required flag (bit 2 of /Ff) marks a field as mandatory before form submission.
Acrobat prevents submission when this field is empty; most parsers expose /Ff correctly.
Gotcha: some tools ignore Required at submission time and allow incomplete forms.
Here: value pre-filled; clearing it and submitting tests viewer enforcement.

## TEXT — Read-Only Field (Ff bit 0 = ReadOnly)

ACC-2026-0001 (read-only)

The ReadOnly flag (bit 0 of /Ff) prevents user editing in the viewer.
The /V value is still fully readable by parsers and included in exported form data.
Gotcha: some tools treat read-only as 'skip during export', losing the value entirely.
Some form designers combine ReadOnly with a coloured background to signal locked content.

## TEXT — Multiline with Scroll Overflow (Ff bit 13 = Multiline)

Line 1: Visible text.
Line 2: Also visible.
Line 3: May be scrolled out of view.
Line 4: Only in /V, not in the AP stream if viewport was too small.
Line 5: Many tools silently drop this line when parsing the appearance stream.

With Multiline set, the field accepts newlines. If text overflows the rect, a scrollbar appears.
The full text lives in /V; the AP stream captures only the lines visible at last save.
Parsing /V gives complete text; parsing the AP stream gives only the visible portion.
PDFBox reads /V correctly. AP-only parsers will silently truncate long multiline values.

## TEXT — Password Field (Ff bit 14 = Password)

GGGGGGGGGGGGGG

The Password flag masks characters with bullets/asterisks in the viewer.
Critical: /V is stored in PLAIN TEXT in the PDF. The masking is display-only.
Beyond document-level encryption, there is NO additional secret for password fields.
Password + Comb simultaneously: spec says Comb is ignored when Password is set. Many generators set both.

## TEXT — Comb Layout (Ff bit 24 = Comb + /MaxLen = 12)

A B C D E 1 2 3 4 5 X Y

Comb divides the widget rectangle into MaxLen equal cells, one character per cell.
Spec requires BOTH the Comb bit AND /MaxLen to be present — if /MaxLen is absent the field is malformed.
MaxLen = 0 with Comb causes division-by-zero in some older libraries (iText 5, ReportLab).
Setting Multiline or Password alongside Comb: spec says Comb is silently ignored in those cases.

## TEXT — Auto-Size Font (/DA with font size 0)

This text should auto-size to fit the field rectangle

Setting font size to 0 in the /DA string tells the viewer to auto-fit the text to the field rect.
Spec: ISO 32000-1 §12.7.3.3. Acrobat honours this for single-line fields.
PDFBox 3.x ignores size=0 and clips text to the BBox instead of shrinking the font.
pdf.js renders text at 0pt — effectively invisible — in this case.

## TEXT — Oversized Font in /DA (font size 72pt in a 20pt-tall field)

Using a font size larger than the widget height is legal per spec but visually clips.
Acrobat clips the overflowing text to the field rect boundary.
PDFBox AP stream generation overflows the BBox — the generated appearance is technically invalid.
Useful as a crash test: any code that divides widget height by font size can get values < 1.

## TEXT — Centre Alignment (/Q = 1)

Centred text value

/Q (Quadding) controls text alignment: 0 = left (default), 1 = centre, 2 = right.
PDFBox honours /Q for PDTextField. Combo box display text ignores /Q in PDFBox 3.x.
Quadding is inheritable: if absent on the field, inherited from the parent or from /AcroForm.
Flat parsers reading only the terminal field's /Q will miss inherited alignment.

## TEXT — Right Alignment (/Q = 2)

1.234,56

Right-aligned text for numeric / currency input patterns.
Required by many financial form designs; ensures numbers right-justify naturally.
Some viewers flip /Q to left-align when the field is in edit mode, reverting on blur.
Works correctly in Acrobat, pdf.js, and PDFBox setQ(2).

### TEXT — RichText Field (Ff bit 26 = RichText, /V + /RV)

Rich text fallback value

With RichText flag (bit 26), /V holds plain-text fallback; /RV holds XHTML-style markup.
Example /RV: <body> Bold </body>
PDFBox setValue() writes /V only — /RV must be set via raw COSDictionary access.
pdf.js ignores /RV entirely and falls back to /V. Acrobat/Reader render /RV markup.

### TEXT — MaxLen Without Comb (/MaxLen = 8, no Comb flag)

ABCDEFGH

/MaxLen caps character input at 8 without the Comb cell visualisation.
If /V already exceeds MaxLen (possible in hand-edited PDFs), Acrobat clips silently.
PDFBox getText() / getValue() returns the full stored /V regardless of MaxLen.
Some validators flag 'value exceeds MaxLen' as an error; spec says it's undefined behaviour.

### TEXT — Invalid Quadding Value (/Q = −1)

Negative /Q = -1

/Q must be 0, 1, or 2 per spec. Negative values exist in wild-crafted and poorly-generated PDFs.
Acrobat normalises /Q < 0 to 0 (left-align). PDFBox stores and returns the raw integer.
Some parsers crash or throw an assertion error when /Q is outside [0..2].
Tests robustness: valid enumerated-field validators must handle out-of-range cleanly.

## CHECKBOXES & RADIO BUTTONS

PDCheckBox / PDRadioButton — Flag and Appearance Quirks

### CHECKBOXES — 6 Visual Themes (Plain / Green / Amber / Red / Gray / Purple)

Plain — no MK, no colour (baseline)

Checked

Green — checked

Checked

Amber — unchecked

Unchecked

Red — checked

Checked

Gray — unchecked (disabled)

Unchecked

Purple — checked

Checked

### CHECKBOX — Standard Checked (/AS = /Yes, off-state = /Off)

Standard checkbox: on-state named /Yes, off-state named /Off (both by convention, not spec mandate).
PDFBox PDCheckBox.check() hardcodes looking for /Yes as the on-state name in the AP dictionary.
If the AP dict uses a non-/Yes on-state (e.g. /On, /Checked), PDBox sets /AS=/Yes but the appearance won't switch — the field appears blank even though it is logically checked.

## CHECKBOX — Custom On-State Name (/AS = /Checked, not /Yes)

✔

The on-state AP stream key is /Checked (not the conventional /Yes).
PDFBox PDCheckBox.check() sets /AS = /Yes — but /Yes has no AP entry — field appears blank in Acrobat.
Your AcroFormTreeBuilder hardcodes the off-state check as '"Off".equalsIgnoreCase(current)'.
Any off-state named /Nein, /false, or /0 will be misidentified as 'checked' by that logic.

## CHECKBOX — Coloured Widget Background (/MK /BG + /BC)

/MK /BG [1 1 0] = yellow background; /MK /BC [0 0 0.8] = blue border colour.
PDFBox reads /MK /BG correctly via PDAppearanceCharacteristicsDictionary.getBackground().
PDFBox does NOT apply /BG when auto-generating AP streams. Background stays white.
You must manually draw the background fill in the AP content stream to honour /BG.

## RADIO — Standard 3-Option Group (single PDRadioButton, 3 widgets)

Option A Option B Option C

A radio group is ONE PDRadioButton field with multiple widget annotations per page.
Each widget represents one option via its AP /N subdictionary key (the export value name).
The field /V holds the currently selected on-state name; all widgets share the same parent.
Parsers that call getWidgets().get(0) to find page/rect report only the first widget's position.

## RADIO — NoToggleToOff CLEARED (Ff bit 15 clear = can deselect)

Yes (click again to deselect) No

Normally clicking a checked radio again does nothing (NoToggleToOff is SET by default).
When NoToggleToOff (bit 15) is CLEARED, clicking the current selection deselects it.
This produces a valid state where /V = /Off and all radios are unchecked simultaneously.
Most UI frameworks hardcode radio buttons as 'always one selected'; this breaks that assumption.

# COMBO BOXES & LIST BOXES

PDComboBox / PDListBox — Edge Cases in Options and Selection

## COMBO BOXES — 6 Visual Themes (Plain / thick / dashed / beveled / inset / underline)

Plain — no MK, no colour (baseline)

Option A

Green — solid, 2.5 pt

Option B

Amber — dashed [4,2]

Option C

Red — beveled (/B)

Option D

Gray — inset (/I)

Option E

Purple — underline (/U)

Option F

## COMBO — Standard Non-Editable (/V must match an /Opt entry)

DE

Standard combo: /V holds the selected export value; /Opt lists allowed values.
Critical: if /V is not in /Opt (e.g. from a prior save), Acrobat shows a blank field.
PDFBox returns the stored /V regardless of whether it matches /Opt — no validation.
Use paired /Opt arrays for export-value ≠ display-name (see next box).

## COMBO — Editable Combo (Ff bit 19 = Edit, user can type custom values)

Custom Tier (not in list)

With the Edit flag set, the user can type a custom value not present in /Opt.
The typed value goes into /V but is NOT added to /Opt — not available in the dropdown on reload.
Some validators incorrectly flag '/V not in /Opt' as an error on editable combos.
Acrobat accepts the custom value; pdf.js clears it on reload.

## COMBO — Paired Options (/Opt = [[export, display], ...])

Germany

/Opt can contain 2-element sub-arrays: [export_value, display_name].
The display name is shown in the UI; the export value is stored in /V and submitted.
PDFBox getOptions() returns display names only — export values are silently dropped.
Round-tripping via the high-level API destroys the export/display pairing.

## LIST — Multi-Select (Ff bit 22 = MultiSelect, /V and /I are arrays)

Low
Medium
High
Critical
Blocker

MultiSelect flag: /V and /I become arrays holding multiple selected values and indices.
/I = [0, 2] means items at index 0 and 2 are selected. /V = [Low, High] holds their export values.
PDFBox ListBox.getValue() handles multi-select; /I-without-/V or out-of-range /I gives wrong results.
Spec requires /I to be sorted ascending. Unsorted /I is common in wild PDFs; Acrobat accepts it.

## LIST — TopIndex Pre-Scrolled (/TI = 4, list opens at item 5)

Item 5
Item 6
Item 7
Item 8
Item 9

/TI (TopIndex) records which item index is at the top of the visible scroll area.
Here /TI = 4 means the list opens scrolled to item 5 ('Item 5') rather than item 1.
PDFBox ignores /TI entirely in both reading and writing.
Round-tripping a document through PDFBox destroys the saved scroll position.

## COMBO — Current Value Not in /Opt (/V = 'CustomEntry', absent from list)

CustomEntry

The stored /V ('CustomEntry') is absent from the /Opt array.
This occurs with editable combos after typing, or in hand-edited PDFs.
Acrobat shows a blank field and may flag a validation warning on submit.
PDFBox getValue() returns the stored /V string without any validation or warning.

# PUSH BUTTONS

PDPushButton — Actions, Print Flags, and Embedded Actions

## PUSH BUTTONS — 6 Visual Themes (Plain / Green / Amber / Red / Gray / Purple)

Plain — no MK, no colour (baseline)

Green — confirm / submit

Amber — warning action

Red — destructive action

Gray — secondary / cancel

Purple — special action

## BUTTON — Reset Form (/A /S /ResetForm)

/ResetForm action resets all fields to their /DV (default value), or blank if /DV is absent.
Optional /Fields array limits which fields are reset; absent = reset ALL fields.
PDFBox has no high-level PDActionResetForm class — requires raw COSDictionary construction.
Empty /Fields = [] resets nothing (a common generator bug). Absent /Fields = reset all.

## BUTTON — Submit Form (/A /S /SubmitForm, Flags = 4 = HTML format)

/SubmitForm action. Flags bit 2 (value 4) = HTML form data format.
Bit 0 = FDF (default, legacy); bit 1 = XFDF; bit 2 = HTML; bit 3 = include empty fields.
pdf.js ignores SubmitForm entirely. Modern servers reject legacy FDF submissions.
PDFBox has PDActionSubmitForm but no execution logic. Parsing the action works.

## BUTTON — Print-Only Widget (F = 36 = NoView | Print)

Annotation flags F = 36 = NoView (bit 6) | Print (bit 3): invisible on screen, prints normally.
Used for print watermarks, invisible tracking fields, and printer-specific form elements.
PDFBox does NOT enforce annotation flags during rendering — this field will be VISIBLE on screen.
pdf.js also renders NoView fields visibly. Only Acrobat Reader correctly hides this.

# FONT EMBEDDING VARIANTS

Embedded Full / Subset / Not-Embedded / Standard-14 — Portability, File Size, and Glyph Availability

## FONT EMBEDDING — Text Field Variants (Plain / Full-embed / Subset / Lato / Helvetica / Courier)

Plain — no MK, inherits AcroForm DA

Plain default (no styling)

FreeSans — fully embedded (all glyphs)

FreeSans fully embedded

FreeSans — subset (only glyphs in value)

Lato Regular — different TTF, full embed

Lato Regular embedded

Helvetica — Standard Type1, NOT embedded

Helvetica (viewer-provided)

Courier — Standard Type1, NOT embedded

Courier (viewer-provided)

## CHECKBOXES — Font Embedding Variants (Plain / No-DA / ZapfDingbats / Lato / 2-glyph subset / Helvetica)

Plain — inherits AcroForm DA

Checked

No /DA — viewer fallback

Checked

ZapfDingbats (/ZaDb) — traditional

Checked

Lato Regular — embedded TTF

Checked

2-glyph subset (only 'l' + ' ')

Checked

Helvetica — Standard Type1

Checked

## TEXT — Value-Only Font Subset (embedded glyphs = exactly the chars in /V)

Font loaded with PDType0Font.load(doc, stream, embedSubset=true).
Before setValue(), font.encode(value) is called to register only the needed glyph IDs.
At doc.save() time, PDFBox subsets the CIDFont to those GIDs only — minimising file size.
If /V is later changed (e.g., by a viewer) to include a new char, that glyph is MISSING.
The subset font cannot render characters beyond those registered at generation time.
Value: "AcroForm subset demo" — unique glyphs: 13 code-points embedded.

# WIDGET ANNOTATION BORDER STYLES

Per-Widget /BS and /MK Dictionary — What Most Viewers Ignore

## WIDGET — Zero-Width Border (/BS /W = 0)

No visible border (W=0)

Zero border width means no visible frame, but the annotation rect remains interaction-active.
Some viewers treat /W = 0 as 'skip AP rendering entirely', making the field completely invisible.
Interactive hit-testing still works — the user can click and type in an invisible field.
Commonly used in form designs that draw their own field box in the page content stream.

## WIDGET — Dashed Border (/BS /S = /D, /D = [4 2])

Dashed border (4 on, 2 off)

Border style /D (dashed) with /D [4 2] = 4pt on, 2pt off.
pdf.js 3.x renders only /S (solid) and /U (underline) — dashed appears as solid.
PDFBox does not generate dashed AP streams — the /BS flag is written but the AP uses a plain rect stroke.
Only Acrobat Reader renders the actual dashed pattern from the /BS /D array.

## WIDGET — Beveled Border (/BS /S = /B) — 3D raised appearance

Beveled border (3D raised)

Beveled style uses lighter/darker shades of the border colour to simulate a 3D raised button.
PDFBox does not generate beveled AP streams — the /BS /S = /B flag is written but rendered as solid.
pdf.js renders as solid. Only Adobe Acrobat shows the true 3D beveled effect.
The shade colours are derived from /MK /BC. If /BC is absent, the effect is undefined.

## WIDGET — Inset Border (/BS /S = /I) — 3D sunken appearance

Inset border (3D sunken)

Inset style is the inverse of Beveled — shading makes the field appear sunken into the page.
Used for read-only or locked field visual distinction in sophisticated form layouts.
Like Beveled, PDFBox does not implement inset AP generation — renders as solid.
Acrobat renders correctly; all other major PDF viewers fall back to solid border.

## WIDGET — Underline Border (/BS /S = /U) — bottom edge only

Underline only border

Underline style draws only the bottom edge of the annotation rect.
Common in flat / minimal form designs; gives a clean 'input line' appearance.
pdf.js supports underline border natively. PDFBox writes the flag but no underline AP.
PDFBox AP stream uses a full rectangle stroke regardless of /S = /U.

### WIDGET — Content Rotated 90° (/MK /R = 90)

Rotate

/MK /R = 90 rotates the field content 90° clockwise within its unchanged rectangle.
The annotation rect stays the same; the text appears rotated inside the widget box.
PDFBox widget.getAppearanceCharacteristics().getRotation() reads /MK /R correctly.
PDFBox AP stream generation IGNORES /R — text appears unrotated despite the flag being set.

### WIDGET — Full /MK Colour Set (/BG orange + /BC blue)

Custom /BG and /BC colours

/MK /BG [1 0.6 0] = orange background; /MK /BC [0 0 0.7] = dark blue border.
PDFBox PDAppearanceCharacteristicsDictionary.getBackground() reads /BG correctly.
PDFBox does NOT apply /BG in auto-generated AP streams; background stays white.
To honour /BG you must manually draw the fill rect in the AP content stream.

## SIGNATURE FIELDS

PDSignatureField — Empty, Lock Dict, and DocMDP Certification

### SIGNATURE — Empty Placeholder (no /V entry)

A signature field with no /V entry is an unsigned placeholder waiting for a signature.
Many parsers crash calling getSignature() on an empty field (NullPointerException).
PDFBox PDSignatureField.getSignature() returns null correctly for unsigned fields.
Downstream code that assumes getSignature() is non-null is a pervasive failure point.

### SIGNATURE — With Lock Dictionary (/Lock /Action /Include)

/Lock dict on a sig field specifies which fields are locked after signing.
/Action = All | Include (lock listed) | Exclude (lock all except listed). /Fields = list.
PDFBox has no high-level SigFieldLock API — raw COSDictionary access only.
Acrobat enforces locks post-signing; all other viewers typically ignore /Lock entirely.

### SIGNATURE — DocMDP Certification (/Reference /TransformMethod /DocMDP, /P = 2)

A certification (DocMDP) signature uses /Reference /TransformParams /P to restrict edits.
/P 1 = no changes; /P 2 = form fill-in allowed; /P 3 = form fill + annotations allowed.
PDFBox signs without reading DocMDP constraints — document can be freely modified after.
Saving a DocMDP-certified PDF with PDDocument.save() (not incremental) invalidates all prior signatures.

# EXTREME EDGE CASES

Legal-but-Unusual PDF Structures That Break Most Parsers

## EDGE CASE — Hidden Widget (annotation F = 2 = Hidden)

Annotation flag F = 2 (Hidden) hides the widget completely — not displayed, not printed.
The /V value still exists and is fully accessible by parsing code.
PDFBox does not enforce F = Hidden during rendering — this widget is VISIBLE on screen.
pdf.js also renders hidden annotation flags visibly. Only Acrobat/Reader hides this.

## EDGE CASE — Zero-Height Widget Rect (/Rect height = 0)

<-- 0px high widget here

Widget with /Rect [x y x y] (height = 0) is technically legal. No visible output.
Code dividing by widget height (font auto-sizing, layout) gets NaN or ArithmeticException.
PDFBox PDRectangle.getHeight() returns 0. Any AP stream content into 0-height BBox is invisible.
Guard: Math.max(1f, rect.getHeight()) before any division. Your PdfFormFieldBuilder does this.

## EDGE CASE — Very Long /V Value (1 000+ characters in a text field)

Line 1: data value row 1.
Line 2: data value row 2.
Line 3: data value row 3.
Line 4: data value row 4.
Line 5: data value row 5.

PDF strings have no spec-defined maximum length. A /V with 1000+ chars is valid.
For multiline fields, PDFBox clips the AP stream to visible lines; /V holds the full text.
JSON serialisation of the full /V may exceed browser message-size or DOM limits.
Some validators cap /V at 32 KB or 65 KB and incorrectly reject valid long values.

## EDGE CASE — Hierarchical Field Name (form.section.person.name)

Nested field value

Field names use dots as hierarchy separators: 'form.section.person.name' = 4 nested nodes.
Each dot level is a PDNonTerminalField. The /T (partial name) of each node has no dot.
A flat field with a literal dot in /T vs a proper nested structure is resolved differently.
FQNs with dots used as HTML attribute values conflict with CSS class selector syntax.

## EDGE CASE — Missing Default Appearance (no /DA on field, no global /DA on AcroForm)

Value with no /DA (will appear blank in most viewers)

When /DA is absent on both the field and the AcroForm, viewers must synthesise a fallback.
Acrobat falls back to Helvetica 12pt. PDFBox generates no AP stream (blank rendering).
NeedAppearances=true + missing /DA: field renders completely blank in pdf.js.
Do not confuse with inherited /DA: a field without /DA inherits from the nearest parent.

## EDGE CASE — /V and /DV Mismatch (current value ≠ default value)

### Changed value (/V)

/DV is the 'default value' restored by a ResetForm action. /V is the current value.
Here /V = 'Changed value' but /DV = 'Original default' — ResetForm will restore to /DV.
PDFBox high-level API has no getDefaultValue() method; requires raw getCOSObject().getString(DA).
Some validators incorrectly flag /V != /DV as a form-integrity error.